

Indian Institute of Technology Dharwad
CS201L: Artificial Intelligence Laboratory
Lab 9 – Regression

1 Problem Statement

You are given the dataset `AirQuality.csv`, which contains various air pollution indicators. Your task is to predict the **CO concentration** (mg/m^3) using regression models.

2 Dataset Details

Input Variables:

- PT08.S1(CO): Tin oxide sensor response (nominally CO targeted)
- NMHC: Non-Methanic HydroCarbons concentration ($\mu g/m^3$)
- C6H6(GT): Benzene concentration ($\mu g/m^3$)
- PT08.S2(NMHC): Titania sensor response (nominally NMHC targeted)
- NOx: NOx concentration (ppb)
- PT08.S3(NOx): Tungsten oxide sensor response (nominally NOx targeted)
- NO2(GT): NO₂ concentration ($\mu g/m^3$)
- PT08.S4(NO2): Tungsten oxide sensor response (nominally NO₂ targeted)
- PT08.S5(O3): Indium oxide sensor response (nominally O₃ targeted)
- T: Temperature (°C)
- RH: Relative Humidity
- AH: Absolute Humidity

Output Variable:

- CO concentration (mg/m^3)

3 Tasks

3.1 Data Splitting

Split the dataset into the following subsets:

- 60% Training Data
- 20% Validation Data
- 20% Test Data

3.2 Correlation Analysis

1. Compute the correlation between each input feature and the target variable **CO concentration** (y).
2. Compare the correlation coefficients and identify the feature that has the **highest absolute correlation** with the target variable.
3. Let this feature be denoted as x_{best} . This feature will be used as the independent variable for the regression tasks.

3.3 Simple Linear Regression

1. Build a simple linear regression model to predict **CO concentration** (y) using the selected feature x_{best} as the independent variable.
2. Plot the **best fit line** on the training dataset.
 - x-axis: Selected Feature (x_{best})
 - y-axis: CO concentration (y)
3. Compute the **RMSE** on the training, validation, and test datasets.

3.4 Polynomial Curve Fitting

1. Build regression models with polynomial degrees

$$p = 2, 3, 4, 5, 6$$

to predict **CO concentration** (y) using the selected feature x_{best} .

2. Compute the **RMSE** on the training and validation datasets for each polynomial degree.
3. Plot **Training RMSE and Validation RMSE vs Polynomial Degree**.
 - x-axis: Polynomial Degree
 - y-axis: RMSE
4. Select the polynomial degree with the **lowest validation RMSE** and evaluate the model on the test dataset.
5. Compute the **RMSE** on the test dataset.

Sample Code Snippets for Regression Tasks

The following code blocks demonstrate how to implement the required correlation analysis, simple linear regression, and polynomial curve fitting using Python and scikit-learn.

1. **Correlation Analysis and Feature Selection:** Compute correlations with the target variable, identify the best feature, and prepare the splits.

```
1 import pandas as pd
2
3 # 1. Compute correlations with the target variable 'CO'
4 correlations = df.corr()['CO'].drop('CO')
5
6 # 2. Find the feature with the highest absolute correlation (x_best)
7 x_best = correlations.abs().idxmax()
8 print(f"Feature with highest absolute correlation: {x_best}")
9
10 # 3. Define independent variables using x_best
11
12 X_train = train_df[[x_best]]
13 y_train = train_df['CO']
14
15 X_val = val_df[[x_best]]
16 y_val = val_df['CO']
17
18 X_test = test_df[[x_best]]
19 y_test = test_df['CO']
```

2. **Simple Linear Regression:** Train a linear model on the selected feature, plot the best fit line, and evaluate using RMSE.

```
1 from sklearn.linear_model import LinearRegression
2 from sklearn.metrics import mean_squared_error
3 import numpy as np
4 import matplotlib.pyplot as plt
5
6 # 1. Build and train the simple linear regression model
7 lin_reg = LinearRegression()
8 lin_reg.fit(X_train, y_train)
9
10 # Generate predictions
11 y_train_pred = lin_reg.predict(X_train)
12 y_val_pred = lin_reg.predict(X_val)
13 y_test_pred = lin_reg.predict(X_test)
14
15 # 3. Compute RMSE for Training, Validation, and Test
16 rmse_train = np.sqrt(mean_squared_error(y_train, y_train_pred))
17 rmse_val = np.sqrt(mean_squared_error(y_val, y_val_pred))
18 rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred))
19 print(f"Train RMSE: {rmse_train:.4f} | Val RMSE: {rmse_val:.4f} | Test RMSE: {rmse_test:.4f}")
```

3. **Polynomial Curve Fitting:** Test different polynomial degrees, plot the RMSE comparisons, and evaluate the best degree on the test set.

```
1 from sklearn.preprocessing import PolynomialFeatures
2
3 degrees = [2, 3, 4, 5, 6]
4 train_rmse = []
5 val_rmse = []
6
7 # 1 & 2. Build models and compute RMSE for each degree
8 for p in degrees:
9     # Transform features
10    poly = PolynomialFeatures(degree=p)
11    X_train_poly = poly.fit_transform(X_train)
12    X_val_poly = poly.transform(X_val)
13
14    # Train model
15    poly_reg = LinearRegression()
16    poly_reg.fit(X_train_poly, y_train)
17
18    # Predict and calculate RMSE
19    train_rmse = np.sqrt(mean_squared_error(y_train, poly_reg.predict(
20    X_train_poly)))
21    val_rmse = np.sqrt(mean_squared_error(y_val, poly_reg.predict(X_val_poly))
22    )
23
24    train_rmse.append(train_rmse)
25    val_rmse.append(val_rmse)
```